

Containerised Research Data Pipelines with Argo Workflows

Alex Lubbock and Piper Fowler-Wright



In this session

- What is Argo Workflows?
- What are its capabilities?
- When should you use it?

Argo Project: *Get stuff done with Kubernetes!*

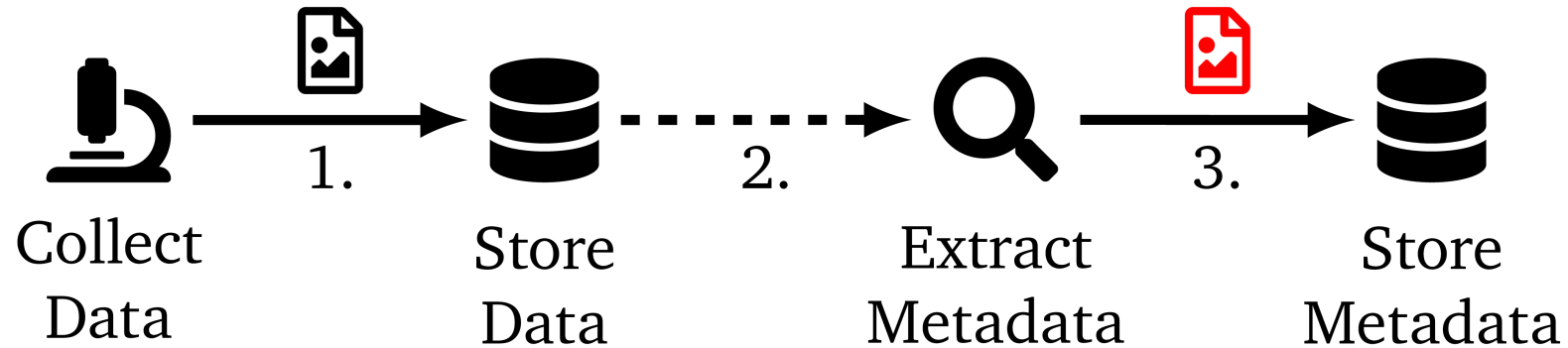
Supporting FAIR Data

Research pipelines need to be:

- **Reproducible**: same inputs, same outputs
- **Traceable**: provenance and metadata captured as part of the run
- **Scalable**: e.g. 15 instruments ~TB/day for 150+ researchers at RFI

Calls for *automated, researcher-driven pipelines on managed infrastructure*

RFI Data Pipeline



1. **Globus API** to mediate data transfer
2. **RFI Metadata API** for metadata extraction
3. **RFI SciCat API** for metadata ingestion (SciCat/MongoDB)

Argo orchestrates 1, 2 & 3 on a timer, user-trigger or event-based sensor.

Workflow Manager Landscape

Tool	Model	Infrastructure
Nextflow	Scientific workflow engine, data-centric	Unmanaged
Snakemake	Scientific workflow engine, file I/O	Unmanaged
Airflow	General workflow orchestrator	Managed application
Globus Flows	Data management platform	Mixed/managed
Argo Workflows	Kubernetes workflow engine, containerised apps	Managed platform

Dependencies between tasks are typically captured using a graph (DAG)



Why Argo Workflows?

- **Kubernetes-native:** workflows are CRDs; the *managed* platform (Kubernetes) handles scheduling, autoscaling and node failures, rather than researchers provisioning and maintaining compute themselves
- **Containerised:** every step runs as a pod, enabling heterogeneous, reproducible workloads
- **Event-driven sensors:** e.g. triggered by AMQP or an S3 bucket operation
- **SSO + RBAC:** researchers submit workflows via SSO; workflows execute with controlled service account permissions

A Workflow Is Just YAML

The RFI pipeline, expressed as an Argo `Workflow` CRD:

1. **Globus API** transfers the data
2. **RFI Metadata API** extracts metadata
3. **RFI SciCat API** ingests it

Each `task` is a pod;
`dependencies` builds the workflow graph (DAG).

```
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: rfi-pipeline-
spec:
  entrypoint: pipeline
  templates:
    - name: pipeline
      dag:
        tasks:
          - name: transfer-data
            template: globus-transfer
          - name: extract-metadata
            dependencies: [transfer-data]
            template: extract-metadata
          - name: ingest-metadata
            dependencies: [extract-metadata]
            template: scicat-ingest
          - name: globus-transfer
            container: {image: rfi/globus-cli}
          - name: extract-metadata
            container: {image: rfi/metadata-extractor}
          - name: scicat-ingest
            container: {image: rfi/scicat-loader}
```

In Practice: The Argo UI

The screenshot displays the Argo UI interface for a workflow named 'example-transfer-t4kdn'. The top navigation bar includes the Argo logo, version 'v3.5.10', and a breadcrumb trail: 'Workflows / scicat-argo-workflows / example-transfer-t4kdn'. On the right, a 'WORKFLOW DETAILS' tab is active. Below the navigation bar, a toolbar contains buttons for 'RESUBMIT', 'DELETE', 'LOGS', and 'SHARE'. A search bar is also present. The main workspace shows a workflow graph with two nodes: 'example-transfer-t4kdn' and 'complete-transfer', both marked with green checkmarks. The right sidebar shows the 'INPUTS/OUTPUTS' tab, which lists parameters and their values.

Parameters	
item_list_filename	{{"destination_path":"/5GB-in-a-single-file/","source_path":"/5GB-in-a-single-file/"}}
item_delete_list_filename	{{"path":"/5GB-in-a-single-file/"}}
transfer_source_collection_name	The Rosalind Franklin Institute Dev Echo Test-Globus-01
transfer_destination_collection_name	The Rosalind Franklin Institute Dev Echo Test-Globus-02
transfer_label	Test submit transfer via ArgoWF - scicat-argo-workflows
transfer_sync_level	2
delete	true

In Practice: SciCat Ingestion

The RFI Metadata Catalog

 Help  About  Piper Fowler-Wright

Datasets /  Add to Cart

Filters 

My data

Public data

Location

Pid

Group

Type

Keyword

Creation Time 

Conditions 

Clear

Apply

Items per page: 100 1 – 100 of 115023 < >

☐	Dataset Name	sourceFolder	size	Creatio...	PID	run#	
☐	Dimitrios Backup	original.zip	0 B	2026-0...	rfi-scat-de...		
☐	20000101_191352	/rfi-ruska-metadata-api-test/20000101_1913...	42 B	2026-0...	undefined/9...		
☐	20000101_190303	/rfi-ruska-metadata-api-test/20000101_1903...	42 B	2026-0...	undefined/8...		
☐	2026-01-04_14-15_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	93 MiB	2026-0...	undefined/4...		
☐	2026-01-04_14-05_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	22 MiB	2026-0...	undefined/4...		
☐	2026-01-04_14-00_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	34 MiB	2026-0...	undefined/ff...		
☐	2026-01-04_14-05_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	99 MiB	2026-0...	undefined/b...		
☐	2026-01-04_12-40_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	196 MiB	2026-0...	undefined/b...		
☐	2026-01-04_12-30_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	37 MiB	2026-0...	undefined/c...		
☐	2026-01-04_12-30_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	177 MiB	2026-0...	undefined/5...		
☐	2026-01-04_12-15_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	27 MiB	2026-0...	undefined/9...		
☐	2026-01-04_12-25_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	69 MiB	2026-0...	undefined/a...		
☐	2026-01-04_11-55_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	61 MiB	2026-0...	undefined/1...		
☐	2026-01-04_12-25_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	70 MiB	2026-0...	undefined/a...		
☐	2026-01-04_12-10_rfi-dorothy-2025-Y-...	/rfi-els-tomography-2025/rfi-dorothy-2025-Y-...	96 MiB	2026-0...	undefined/5...		

Handling Failures & Human Oversight

- **Automatic retries:** transient failures (flaky APIs, network blips) retry with backoff
- **Notifications:** `onExit` calls a webhook, e.g. Slack or email
- **Suspend steps:** a workflow can pause and wait for manual approval
- **Resubmission:** failed runs are resubmitted from the UI or CLI without re-running steps that already succeeded
- Trade-off: flexibility adds **operational complexity**; debugging can span workflows, pods, containers and Kubernetes events

Argo Workflows: Takeaways

- Run heterogeneous, container-based workloads on your cluster
- K8s roles and SSO for functional and user-level access, with web UI
- **WorkflowTemplates**: reusable, parameterised pipelines researchers submit via UI forms, no YAML required
- **Argo Events + Sensors**: trigger runs from webhooks, queues or cron, not just manual submission
- **Workflow archive**: completed runs, specs and logs persist for audit/provenance and FAIR traceability
- Artifact support (S3, GCS, Git, HTTP...), metrics, retry handling and more...